Date __/__/____
Page _____

Module - 2

Syllabus:

Syntax Analysis: Introduction, Context Free Grammars, Writing a grammar, Top-Down Parsers, Bottom-Up Parsers.

Textbook 2: Chapter 4: 4.1, 4.2, 4.3, 4.4, 4.5

Chapter - 4
Syntax Analysis

4.1 Introduction:-

4.1.1 The Role of the Parser:-

Definition:- Parsing is the process of getting tokens from the lexical analyzer and obtains a derivation for the sequence of tokens and builds a parse tree.

* Thus, if the program is syntactically correct, the parse tree is generated. If a derivation for the sequence of tokens does not exist i.e, if the program is syntactically wrong, it results in syntax error and the parser displays the appropriate error messages.
* The parse tree is also called syntax tree.
* Parser also called syntax analyzer is the one which does parsing.

* The role of the parser or the various activities that are peformed by the parser are as follows:
1. Parser reads sequence of tokens from the lexical analyzer.
2. The parser checks whether the tokens obtained from lexical analyzer can be succesfully generated. This is done by obtaining a derivation for a sequence of tokens and builds the parse tree.

3. If a derivation is obtained using the sequence of tokens it indicates that program is syntactically correct and the parse tree is generated.
4. If a derivation is not obtained using the sequence of tokens it indicates that program is syntactically wrong and the parse tree is not generated. The parser reports appropriate error messages clearly and accurately along with line numbers.
5. Parser also recovers from each error quickly so that subsequent errors can be detected and displayed so that the user can correct the program.

* The block diagram shows the interaction of parser with other modules and phases:

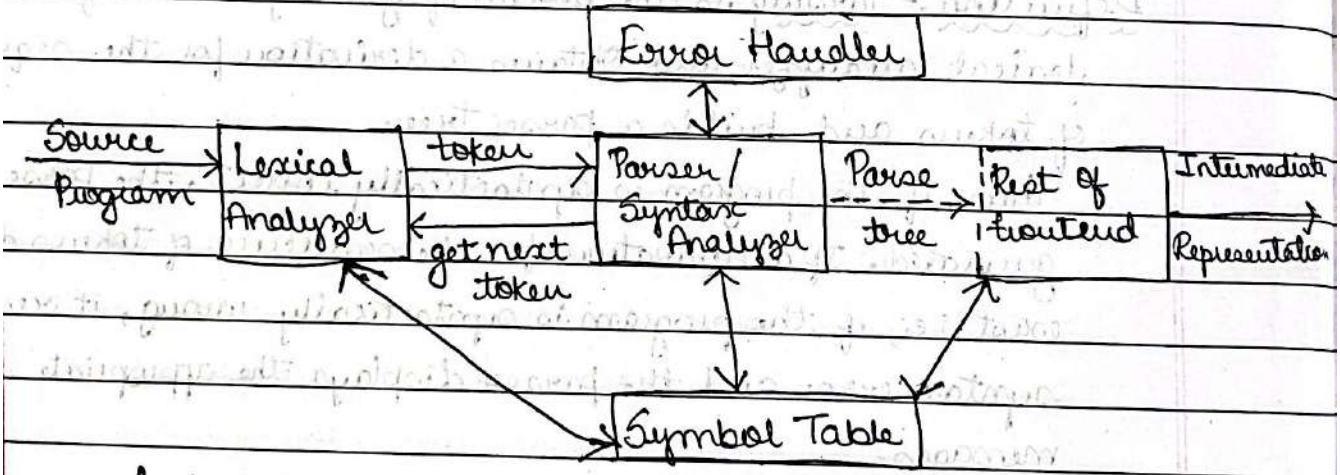


fig 4.1: Position of parser in compiler model

* There are three general types of parsers for grammars: Universal, top-down, and bottom-up.
* Universal parsing methods such as the Cocke-Younger-Kasami algorithm and Earley's algorithm can parse any grammar. These methods are in-efficient.
* The methods commonly used in compilers can be classified as being either top-down or bottom-up.

* Top-down methods build parse trees from the top (root) to the bottom (leaves), while bottom-up methods start from the leaves and work their way up to the root. In either case, the input to the parser is scanned from left to right, one symbol at a time.

4.1.2 Representative Grammars :-

* Some of the grammars that will be examined in this Chapter are presented here for ease of reference.
* Constructs that begin with keywords like while or int, are relatively easy to parse, because the keyword guides the choice of the grammar production that must be applied to match the input.
* Associativity and precedence are captured in the following grammar,

$$E \rightarrow E + T \mid T$$
$$T \rightarrow T * F \mid F$$
$$F \rightarrow (E) \mid id \quad \ldots\ldots (4.1)$$

* 'E' represents 'expressions' consisting of 'terms' seperated by '+' signs, 'T' represents 'terms' consisting of 'factors' seperated by '*' signs, and 'F' represents factors that can be either 'parenthesized expressions' or 'identifiers'.

* Expression grammer (4.1) belongs to the class of LR grammars that are suitable for bottom-up parsing.
* This grammar can be adapted to handle additional operators and additional levels of precedence. However, it cannot be used for top-down parsing, because it is left recursive.

* The following non-left-recursive variant of the expression grammar (4.1) will be used for top-down parsing:

$$E \rightarrow TE'$$
$$E' \rightarrow +TE' \mid \epsilon$$
$$T \rightarrow FT'$$
$$T' \rightarrow *FT' \mid \epsilon \quad \ldots\ldots (4.2)$$
$$F \rightarrow (E) \mid id$$

The following grammar treats '+' and '' alike, so it is useful for illustrating techniques for handling ambiguities during parsing:

$$E \rightarrow E+E \mid E*E \mid (E) \mid id \quad \ldots\ldots (4.3)$$

Here, 'E' represents expressions of all types. Grammar (4.3) permits more than one parse tree for expressions like a+b*c.

4.1.3 Syntax Error Handling:-

* A compiler is expected to assist the programmer in locating and tracking down errors that inevitably creep into programs, despite the programmer's best efforts.
* Most programming language specifications do not describe how a compiler should respond to errors; error handling is left to the compiler design.
* Common programming errors can occur at many different levels:
1. Lexical errors: include misspellings of identifiers, keywords, or operators - e.g., the use of an identifier 'elipseSize' instead of 'ellipseSize' - and missing quotes around text intended as a string.

2. Syntactic errors: include misplaced semicolons or extra or missing braces; that is, '{' or '}'. Another example, in C or Java the appearance of a 'case' statement without enclosing 'switch' is a syntactic error.

3. Semantic errors: include type mismatches between operators and operands, e.g., the return of a value in a Java method with result type 'void'.

4. Logical errors: can be anything from incorrect reasoning on the part of the programmer to the use in C program of assignment operator '=' instead of comparison operator '=='.

* The precision of parsing methods allows syntactic errors to be detected very efficiently.
* Another reason for emphasizing error recovery during parsing is that errors appear syntactic, whatever their cause, and are exposed when parsing cannot continue.
* A few semantic errors, such as type mismatches, can also be detected efficiently; however, accurate detection of semantic and logical errors at compile time is in general a difficult task.

* The error handler in a parser has goals that are simple to state but challenging to realize:
1. Report the presence of errors clearly and accurately.
2. Recover from each error quickly enough to detect subsequent errors.
3. Add minimal overhead to the processing of correct programs.

4.1.4 Error-Recovery Strategies:-

* When an error is detected, the simplest approach for the parser is to quit with an informative error message when it detects the first error. Additional errors are often uncovered if the parser can restore itself to a state where processing of input can continue with reasonable hopes that the further processing will

provide meaningful diagnostic information.

* If errors pile up, it is better for the compiler to give up after exceeding some error limit than to produce an annoying avalanche of 'spurious' errors.

* The various error recovery strategies are:
1. Panic-Mode Recovery
2. Error Productions
3. Phrase-level Recovery
4. Global Correction

1. Panic-Mode Recovery:- It is the simplest and most popular error recovery method.

*When an error is detected, the parser discards symbols one at a time until next valid token (synchronizing token) is found.

* The synchronizing tokens are usually delimiters, such as semicolon or '}', whose role in the source program is clear and unambiguous.

* The compiler designer must select the synchronizing tokens appropriate for the source language.

*While panic-mode correction often skips a considerable amount of input without checking it for additional errors, it has the advantage of simplicity, and, unlike some methods to be considered later, is guaranteed not to go into an infinite loop.

2. Error Productions:- By anticipating common errors that might be encountered, we can augment the grammar for the language at hand with productions that generate erroneous constructs.

* A parser constructed from a grammar augmented by these error productions detects the anticipated errors when an error production is used during parsing.

* The parser can then generate appropriate error diagnostics about the erroneous construct that has been recognized in the input.

Disadvantages:-

1. Can resolve many errors, but not all potential errors.
2. The introduction of error productions will complicate the grammar.

3. Phrase-Level Recovery:- On discovering an error, a parser may perform local correction on the remaining input; i.e., it may replace a prefix of remaining input by some string that allows the parser to continue.

* A typical local correction is to replace a comma by a semicolon, delete an extraneous semicolon, or insert a missing semicolon.

Disadvantages:-

1. Very difficult to implement.
2. Slows down the parsing of correct programs.
3. Proper care must be taken while choosing replacements as they may lead to infinite loops.

4. Global Correction:- These methods replace incorrect input with correct input string using least-cost-correction algorithms.

* These algorithms take an incorrect input string x and grammar G, and find a parse tree for a related string y, such that the number of insertions, deletions and changes of tokens required to transform x into y is as small as possible.
* These methods are costly to implement in terms of time and space and hence are only of theoretical interest.

4.2 Context-Free Grammars:-

4.2.1 The Formal Definition of a Context-Free Grammar:-

Definition:- The context-free grammar in short a CFG is 4 tuple, $G = (V, T, P, S)$ where,

$V \rightarrow$ is set of variables. The variables are also called non-terminals

$T \rightarrow$ is set of terminals.

$P \rightarrow$ is set of productions. All productions in P are of the form $A \rightarrow \alpha$ where A is non-terminal and α is string of grammar symbols.

$S \rightarrow$ is the start symbol

Eg:- stmt $\rightarrow$ if (expr) stmt else stmt .... (4.4)

1. Terminals are the basic symbols from which strings are formed. The term 'token name' is a synonym for 'terminal' and frequently we will use the word 'token' for terminal when it is clear that we are talking about just token name. In (4.4), the terminals are the keywords 'if' and 'else' and by the symbols '(' and ')'.

2. Non-terminals are syntactic variables that denote sets of strings. In (4.4), 'stmt' and 'expr' are nonterminals. The sets of strings denoted by nonterminals help define the language generated by the grammar. Nonterminals impose a hierarchical structure on the language that is key to syntax analysis and translation.

3. In a grammar, one non-terminal is distinguished as the start symbol, and the set of strings it denotes is the language generated by the grammar.

4. The productions of a grammar specify the manner in which the terminals and nonterminals can be combined to form strings. Each production consists of:

(a) A nonterminal called the head or left side of the production; this production defines some of the strings denoted by the head.

(b) The symbol $\rightarrow$. Sometimes ::= has been used in place of the arrow.

(c) A body or right side consisting of zero or more terminals and nonterminals. The components of the body describe one way in which strings of the nonterminal at the head can be constructed.

4.2.2 Notational Conventions:-

1. These symbols are terminals:
   (a) Digits from 0 to 9.
   (b) Operator symbols such as +, *, -, /, etc.
   (c) Lower case letters near the beginning of alphabets such as a, b, c, d, etc.
   (d) Punctuation symbols such as parentheses, comma, and so on.
   (e) Boldface strings such as 'id' or 'if', each of which represents a single terminal symbol.

2. These symbols are nonterminals:
   (a) Uppercase letters in the alphabet, such as A, B, C.
   (b) The letter S, which, when it appears, is usually the start symbol.
   (c) Lowercase, italic names such as 'expr' or 'stmt',

3. Uppercase letters in the alphabet, such as, X, Y, Z, represent grammar symbols; that is, either nonterminals or terminals.

4. Lowercase letters in the alphabet, u, v, ..., z, represent (possibly empty) strings of terminals.

5. Lowercase Greek letters, α, β, γ for example, represent strings of grammar symbols. Thus, a generic production can be written as $A \to \alpha$, where A is the head and α the body.

6. A set of productions $A \to \alpha_1, A \to \alpha_2, \ldots A \to \alpha_k$ with a common head A (call them A-productions), may be written $A \to \alpha_1 | \alpha_2 | \cdots | \alpha_k$. Call $\alpha_1, \alpha_2, \ldots, \alpha_k$ the alternatives for A.

7. Unless stated otherwise, the head of the first production is the start symbol.

4.2.3 Derivations:-

Definition: The process of obtaining string of terminals and/or non-terminals from the start symbol by applying some set of production (or all productions) is called derivation.

For example, if $A \to \alpha B \gamma$ and $B \to \beta$ are the productions, the string $\alpha\beta\gamma$ can be obtained from A-production as shown:

$A \Rightarrow \alpha B \gamma$ [Apply the production $A \to \alpha B \gamma$]
$\Rightarrow \alpha \beta \gamma$ [Replace B by β using the production $B \to \beta$]

The above derivation can also be written as:
$A \overset{+}{\Rightarrow} \alpha\beta\gamma$

(Note: $\overset{+}{\Rightarrow}$ means, 'derives in one or more steps.')
→ $\overset{*}{\Rightarrow}$ means, zero or more productions are applied.
→ If a string is obtained by applying only one production, then it is called one-step derivation and is denoted by the symbol '$\Rightarrow$'.

Example :- Consider the grammar shown below from which any arithmetic expression can be obtained.

$E \rightarrow E + E$

$E \rightarrow E - E$

$E \rightarrow E * E$

$E \rightarrow E / E$

$E \rightarrow id$

Obtain the string id + id * id and show the derivation for same.

Solution :-

$E \Rightarrow E + E$

$\Rightarrow id + E$

$\Rightarrow id + E * E$

$\Rightarrow id + id * E$

$\Rightarrow id + id * id$

Thus, the above sequence of steps can also be written as,

$E \overset{+}{\Rightarrow} id + id * id$

* The two types of derivations are :

1. Leftmost derivation (LMD)
2. Rightmost derivation (RMD)

1. Leftmost derivation :- The process of obtaining a string of terminals from the sequence of replacements such that only leftmost non-terminal is replaced at each and every step is called leftmost derivation.

Example :- $E \rightarrow E + E$

$E \rightarrow E * E$

$E \rightarrow (E)$

$E \rightarrow id$

Obtain string : id + id * id

Solution : $E \underset{lm}{\Rightarrow} E + E$

$\Rightarrow id + E$

$\Rightarrow id + E * E$

$\Rightarrow id + id * E$

$\Rightarrow id + id * id$

2. Rightmost derivation:- The process of obtaining a string of terminals from a sequence of replacements such that only rightmost non-terminal is replaced at each and every step is called rightmost derivation.

Example:- $E \to E+E$	$E \underset{rm}{\Rightarrow} E + E$
$E \to E*E$	$\Rightarrow E + E * E$
$E \to (E)$	$\Rightarrow E + E * id$
$E \to id$	$\Rightarrow E + id * id$
Obtain string: id + id * id	$\Rightarrow id + id * id$

* Sentential Form of G:- (or sentence of G).

→ If $S \overset{*}{\Rightarrow} \alpha$, where S is the start symbol of grammar G, we say that α is a sentential form of G. The sentential form may contain both terminals and non-terminals, and may be empty.

→ A sentence of G is a sentential form with no nonterminals.

→ The language generated by a grammar is its set of sentences. Thus, a string of terminals w is in L(G), the language generated by G, if and only if w is a sentence of G (or $S \overset{*}{\Rightarrow} w$). A language that can be generated by a grammar is context-free grammar.

Example:- The final string of terminals obtained, i.e., id + id * id is called sentence of the grammar.

* Types of sentential form:-

1. Left sentential form
2. Right sentential form

1. Left-sentential form:- If there is a derivation of the form $S \overset{*}{\Rightarrow} \alpha$, where at each step in the derivation process only a left most variable is replaced, then α is called left-sentential form of G.

Example:- Consider the following grammar and leftmost derivation:

Grammar	Leftmost derivation
$E \to E + E$	$E \underset{lm}{\Rightarrow} E + E$
$E \to E * E$	$\Rightarrow id + E$
$E \to (E)$	$\Rightarrow id + E * E$
$E \to id$	$\Rightarrow id + id * E$
	$\Rightarrow id + id * id$

In the above leftmost derivation, the string of grammar symbols obtained in each step such as:

{ E + E, id + E, id + E * E, id + id * E, id + id * id }

are the various left-sentential forms of given grammar.

2. Right-sentential form:- If there is a derivation of the form $S \overset{*}{\Rightarrow} \alpha$, where at each step in the derivation process only a rightmost non-terminal is replaced, then α is called right-sentential form of G.

Example:- Consider the grammar and its rightmost derivation:

Grammar:	Rightmost derivation
$E \to E + E$	$E \underset{rm}{\Rightarrow} E + E$
$E \to E * E$	$\Rightarrow E + E * E$
$E \to (E)$	$\Rightarrow E + E * id$
$E \to id$	$\Rightarrow E + id * id$
	$\Rightarrow id + id * id$

In the above rightmost derivation, string of grammar symbols obtained are: { E+E, E+E*E, E+E*id, E+id*id, id+id*id } are the various right-sentential forms of given grammar.

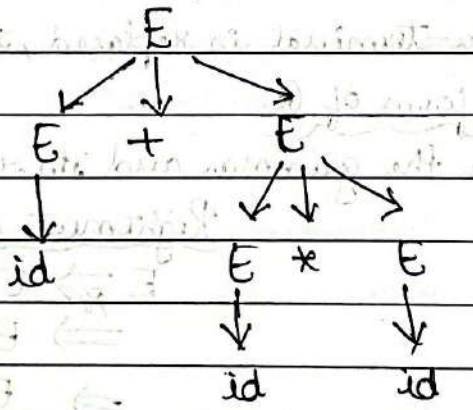
4.2.4 Parse Trees and Derivations :-

* A parse tree is a graphical representation of a derivation that filters out the order in which productions are applied to replace nonterminals.
* Each interior node of a parse tree represents the application of a production.
* The interior node is labeled with the nonterminal 'A' in the head of the production; the children of the node are labeled, from left to right, by the symbols in the body of the production by which this A was replaced during the derivation.

Example :- Consider the grammar and its rightmost derivation:

Grammar	Rightmost derivation
$E \to E+E$	$E \underset{rm}{\Rightarrow} E+E$
$E \to E*E$	$\Rightarrow E+E*E$
$E \to (E)$	$\Rightarrow E+E*id$
$E \to id$	$\Rightarrow E+id*id$
	$\Rightarrow id+id*id$

Parse tree :-



* The leaves of a parse tree are labeled by nonterminals or terminals, and, read from left to right, constitute a sentential form, called the yield or frontier of the tree.
* Read from left to right :→ id + id * id, this is the yield of the given parse tree.

4.2.5 Ambiguity :-

* A grammar that produces more than one parse tree for some sentence is said to be ambiguous.
* An ambiguous grammar is one that produces more than one leftmost derivation or more than one rightmost derivation for the same sentence.

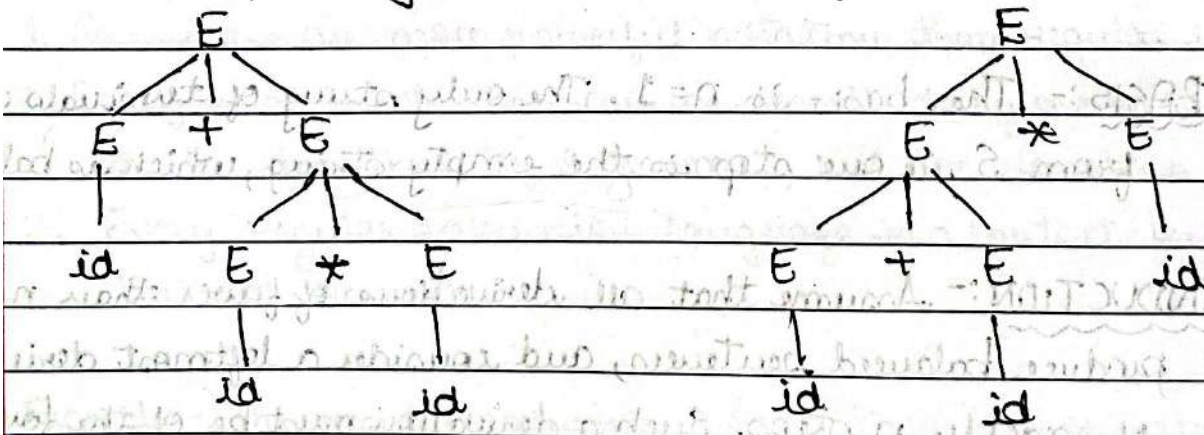
Example :- The arithmetic expression grammar below permits two distinct leftmost derivations for the sentence id + id * id.

$E \rightarrow E+E \mid E*E \mid (E) \mid id$

Leftmost derivation :-

	(a)	
$E \Rightarrow E + E$		$E \Rightarrow E * E$
$\Rightarrow id + E$		$\Rightarrow E + E * E$
$\Rightarrow id + E * E$		$\Rightarrow id + E * E$
$\Rightarrow id + id * E$		$\Rightarrow id + id * E$
$\Rightarrow id + id * id$		$\Rightarrow id + id * id$

The corresponding parse trees are as follows :



* Since the two parse trees are different for the same sentence id + id * id by applying leftmost derivation, the grammar is ambiguous. It is convenient to use carefully chosen ambiguous grammars, together with disambiguating rules that 'throw away' undesirable parse trees, leaving only one tree for each sentence.

4.2.6 Verifying the Language Generated by a Grammar:-

* Although compiler designers rarely complete programming-language grammar, it is useful to be able to reason that a given set of productions generates a particular language.
* A proof that a grammar G generates a language L has two parts: show that every string generated by G is in L, and conversely that every string in L can indeed be generated by G.

Example:- Consider the following grammar:

$$S \rightarrow (S)\,S \mid \epsilon \quad \ldots (4.13)$$

* This grammar generates all strings of balanced parentheses, and only such strings, because every sentence derivable from S is balanced, and then that every balanced string is derivable from S.
* To show that every sentences derivable from S is balanced, we use an inductive proof on the number of steps n in a derivation.

BASIS:- The basis is $n = 1$. The only string of terminals derivable from S in one step is the empty string, which is balanced.

INDUCTION:- Assume that all derivations of fewer than n steps produce balanced sentences, and consider a leftmost derivation of exactly n steps. Such a derivation must be of the form:

$$S \underset{lm}{\Rightarrow} (S)\,S \underset{lm}{\overset{*}{\Rightarrow}} (x)S \underset{lm}{\overset{*}{\Rightarrow}} (x)y$$

* The derivations of x and y from S take fewer than n steps, so by the induction hypothesis x and y are balanced.
* Therefore, the string (x)y must be balanced, i.e., equal number of left and right parentheses, and every prefix has atleast as many left parentheses as right.

* Now, we must show that every balanced string is derivable from S. To do this, use induction on the length of a string.

BASIS:- If the string is of length 0, it must be ϵ, which is balanced.

INDUCTION:- First, Observe that every balanced string has even length. Assume that every balanced string of length less than '2n' is derivable from S, and consider a balanced string 'w' of length '2n', $n \geq 1$. Surely 'w' begins with a left parenthesis.

* Let (x) be the shortest non-empty prefix of 'w' having an equal number of left and right parentheses.
* Then w can be written as w = (x)y where both x and y are balanced. Since x and y are of length less than 2n, they are derivable from S by the inductive hypothesis.
* Thus, we can find a derivation of the form:

$$S \Rightarrow (S)S \overset{*}{\Rightarrow} (x)S \overset{*}{\Rightarrow} (x)y$$

proving that w = (x)y is also derivable from S.

4.2.7 Context-Free Grammar Versus Regular Expressions:-

1. Grammars are more powerful notation than regular expressions.
2. Every construct that can be described by a regular expression can be described by a grammar, but not vice-versa.
3. Every regular expression language is a context-free language, but not vice-versa.

Example:- The regular expression $(a|b)^*abb$ and the grammar

$$A_0 \rightarrow aA_0 \mid bA_0 \mid aA_1$$
$$A_1 \rightarrow bA_2$$
$$A_2 \rightarrow bA_3$$
$$A_3 \rightarrow \epsilon$$

describe the same language, the set of strings of a's and b's ending in abb.

* We can construct mechanically a grammar to recognize the same language as a nondeterministic finite automata (NFA).
* The grammar above is constructed from NFA using the following construction:
1. For each state i of the NFA, create a nonterminal Ai.
2. If state i has a transition to state j on input 'a', add the production Ai → a Aj. If state i goes to state j on input ε, add the production Ai → Aj.
3. If i is an accepting state, add Ai → ε.
4. If i is the start state, make Ai be the start symbol of the grammar.

* On the other hand, the language $L = \{a^n b^n \mid n \geq 1\}$ with an equal number of a's and b's is a prototypical example of a language that can be described by a grammar but not by a regular expression.
* Suppose, L were the language defined by some regular expression. We could construct a DFA D with a finite number of states, say k, to accept L.
* Since D has only k states, for an input beginning with more than k a's, D must enter some state twice, say s_i, as in fig 4.6.
* Suppose that the path from s_i back to itself is labeled with a sequence a^{j-i}. Since $a^i b^i$ is in language, there must be a path labeled b^i from s_i to an accepting state f.
* But, then there is also a path from the initial state 's0' through 'si' to 'f' labeled $a^j b^i$, as in fig 4.6.
* Thus, D also accepts $a^j b^i$, which is not the language, contradicting the assumption that L is the language accepted by D.

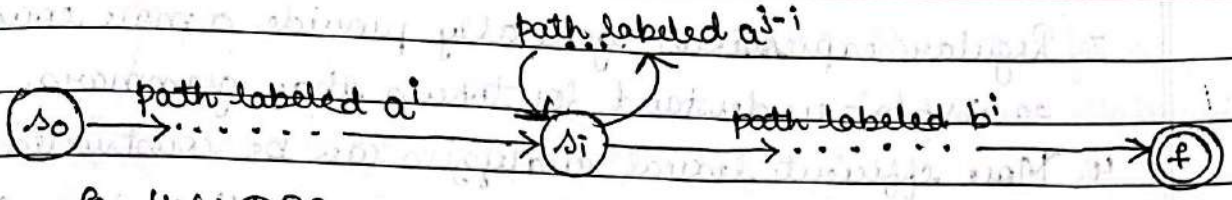


Fig 4.6: DFA D accepting both a^ib^i and a^jb^j.

* Colloquially, we say that 'finite automata cannot count', meaning that a finite automaton cannot accept a language like $\{a^nb^n \mid n \geq 1\}$ that would require it to keep count of the number of a's before it see's the b's.
* Likewise, "a grammar can count two items but not three".

4.3 Writing a Grammar:-

* Grammars are capable of describing most, but not all, of the syntax of programming languages. For instance, the requirement that identifiers be declared before they are used, cannot be described by a context-free grammar.
* Therefore, the sequences of tokens accepted by a parser form a superset of the programming-language; subsequent phases of the compiler must analyze the output of the parser to ensure compliance with rules that are not checked by the parser.

4.3.1 Lexical Versus Syntactic Analysis:-

* To use regular expressions to define the lexical syntax of a language, there are several reasons.

1. Separating the syntactic structure of a language into lexical and non-lexical parts provides a convenient way of modularizing the front end of a compiler into two manageable-sized components.
2. The lexical rules of a language are frequently quite simple, and to describe them we do not need a notation as powerful as grammars.

3. Regular expressions generally provide a more concise and easier-to-understand for tokens than grammars.
4. More efficient lexical analyzers can be constructed automatically from regular expressions than from arbitrary grammars.

* Regular expressions are most useful for describing the structure of constructs such as identifiers, constants, keywords, and white space.
* Grammars are most useful for describing nested structures such as balanced parentheses, matching begin-end's, corresponding if-then-else's, and so on. These nested structures cannot be described by regular expressions.

4.3.2 Eliminating Ambiguity :-

* Sometimes an ambiguous grammar can be rewritten to eliminate the ambiguity.

Example:- Eliminate the ambiguity from the following 'dangling-else' grammar:

stmt → if 'expr' then 'stmt'
| if 'expr' then 'stmt' else 'stmt'
| other ... (4.14)

* Here, 'Other' stands for any other statement.
* According to this grammar, the compound conditional statement has the parse tree shown in Fig. 4.8.

if E_1 then S_1 else if E_2 then S_2 else S_3

* Grammar (4.14) is ambiguous since the string below has two parse trees shown in fig. 4.9.

if E_1 then if E_2 then S_1 else S_2 ... (4.15)

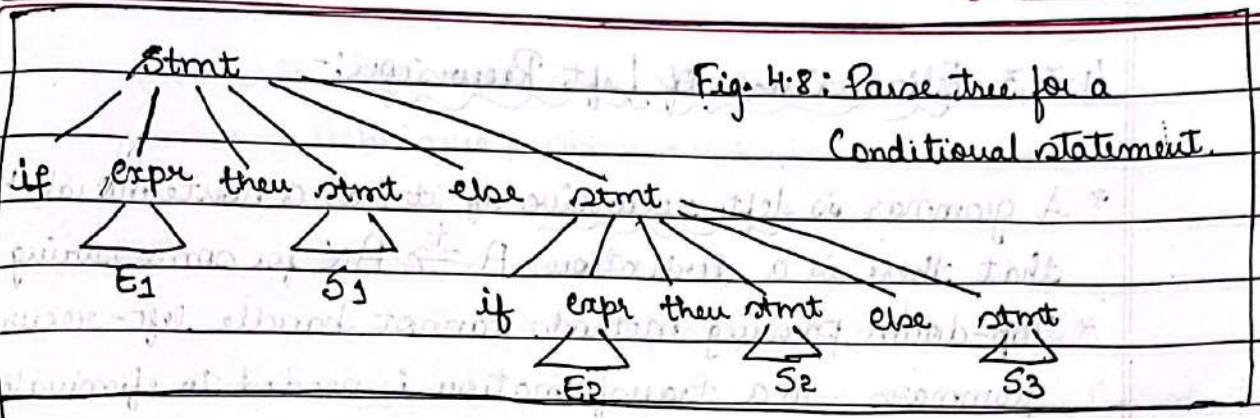


Fig. 4.8: Parse tree for a Conditional statement.

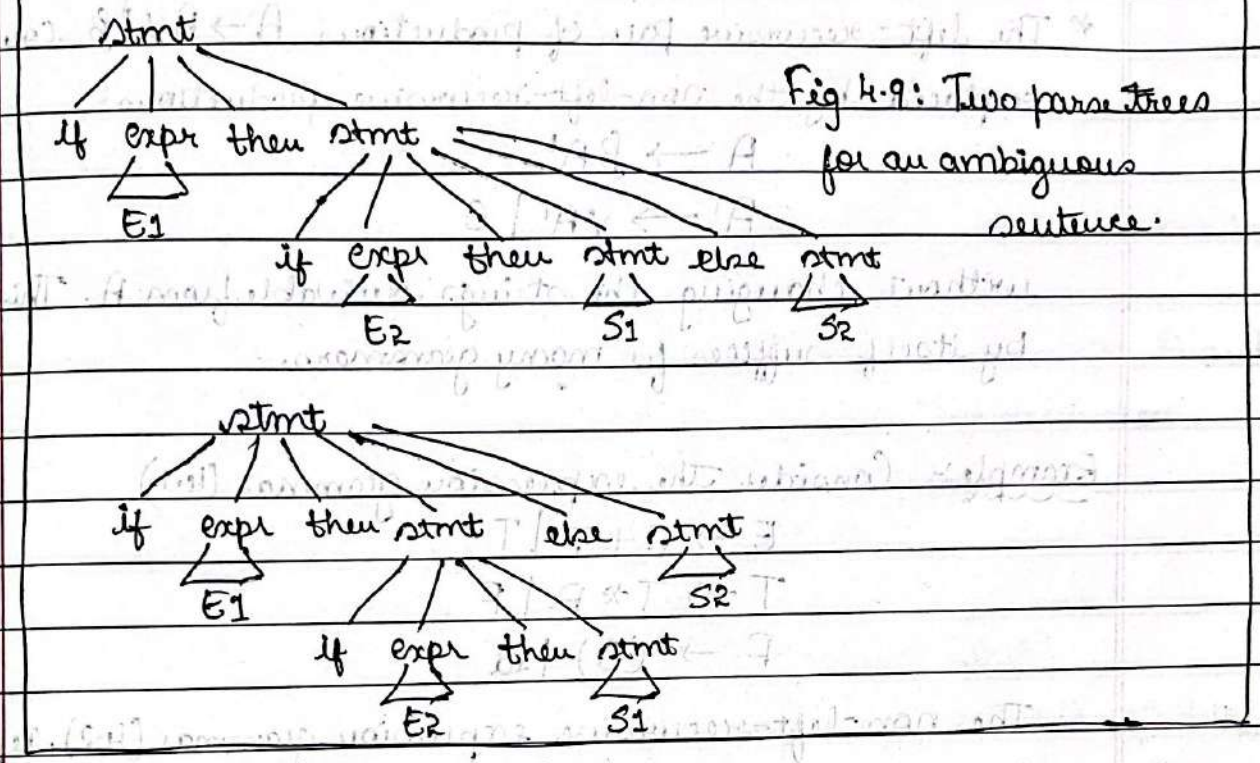


Fig 4.9: Two parse trees for an ambiguous sentence.

* In all programming languages with conditional statements of this form, the first parse tree in fig 4.9, is preferred.
* The general rule is, 'Match each else with the closest unmatched then'. This dis-ambiguating rule can theoretically be incorporated directly into a grammar, but in practice it is rarely built into the productions.

4.3.3 Elimination of Left Recursion:-

* A grammar is left recursive if it has a nonterminal A such that there is a derivation $A \xrightarrow{+} A\alpha$ for some string α.
* Top-down parsing methods cannot handle left-recursive grammars, so a transformation is needed to eliminate left recursion.
* The left-recursive pair of productions $A \to A\alpha \mid \beta$ could be replaced by the non-left-recursive productions:

$$A \to \beta A'$$
$$A' \to \alpha A' \mid \epsilon$$

without changing the strings derivable from A. This rule by itself suffices for many grammars.

Example:- Consider the expression grammar (4.1)

$$E \to E + T \mid T$$
$$T \to T * F \mid F$$
$$F \to (E) \mid id$$

The non-left-recursive expression grammar (4.2), repeated here,

$$E \to TE'$$
$$E' \to +TE' \mid \epsilon$$
$$T \to FT'$$
$$T' \to *FT' \mid \epsilon$$
$$F \to (E) \mid id$$

is obtained by eliminating immediate left recursion from the expression grammar (4.1).

* The left-recursive pair of productions $E \to E+T \mid T$ are replaced by $E \to TE'$ and $E' \to +TE' \mid \epsilon$.
* The new productions for T and T' are obtained similarly by eliminating immediate left recursion.

* Immediate left recursion can be eliminated by the following technique, which works for any number of A-productions.

* First, group the productions as:

$$A \to A\alpha_1 \mid A\alpha_2 \mid \cdots \mid A\alpha_m \mid \beta_1 \mid \beta_2 \mid \cdots \mid \beta_n$$

where no β_i begins with an A. Then, replace the A-productions by,

$$A \to \beta_1 A' \mid \beta_2 A' \mid \cdots \mid \beta_n A'$$
$$A' \to \alpha_1 A' \mid \alpha_2 A' \mid \cdots \mid \alpha_m A' \mid \epsilon$$

* The nonterminal A generates the same strings as before but is no longer left recursive.

* This procedure eliminates all left recursion from the A and A' productions, but it does not eliminate left recursion involving derivations of two or more steps.

Example:- Consider the grammar

$$S \to Aa \mid b$$
$$A \to Ac \mid Sd \mid \epsilon \qquad \cdots (4.18)$$

The nonterminal S is left recursive because $S \Rightarrow Aa \Rightarrow Sda$, but it is not immediate left recursive.

* Algorithm for eliminating left recursion, systematically eliminates left recursion from a grammar.

* It is guaranteed to work if the grammar has no cycles (derivations of the form $A \overset{+}{\Rightarrow} A$) or ϵ-productions (productions of the form $A \to \epsilon$).

* Cycles can be eliminated systematically from a grammar, as can ϵ-productions.

Algorithm: Eliminating left recursion.

INPUT: Grammar G with no cycles or ε-productions.
OUTPUT: An equivalent grammar with no left recursion.

METHOD: Apply the algorithm to G. Note that the resulting non-left-recursive grammar may have ε-productions.

1. arrange the nonterminals in some order $A_1, A_2, \ldots, A_n$.
2. for (each i from 1 to n) {
3. for (each j from 1 to i-1) {
4. replace each production of the form $A_i \to A_j \gamma$ by the productions $A_i \to \delta_1 \gamma \mid \delta_2 \gamma \mid \cdots \mid \delta_k \gamma$, where $A_j \to \delta_1 \mid \delta_2 \mid \cdots \mid \delta_k$ are all current A_j-productions
5. }
6. eliminate the immediate left recursion among the A_i-productions
7. }

* The procedure for the above algorithm works as follows.
* In the first iteration for i = 1, the outer for-loop of lines (2) through (7) eliminates any immediate left recursion among A_1-productions.
* Any remaining A_2 productions of the form $A_1 \to A_l \alpha$ must therefore have $l > 1$. After the $i-1^{st}$ iteration of outer for-loop, all nonterminals A_k, where $k < i$, are 'cleaned'; i.e., any production $A_k \to A_l \alpha$, must have $l > k$.
* As a result, on the i^{th} iteration, the inner loop of lines (3) through (5) progressively raises the lower limit in any production $A_i \to A_m \alpha$, until we have $m \geq i$. Then, eliminating immediate left recursion for the A_i productions at line (6) forces m to be greater than i.

4.3.4 Left - Factoring :-

* Left factoring is a grammar transformation that is useful for producing a grammar suitable for predictive, or top-down, parsing.
* When the choice between two alternative A-productions is not clear, we may be able to rewrite the productions to defer the decision until enough of the input has been seen that we can make the right choice.

Example :- If we have two productions

stmt ⟶ if 'expr' then 'stmt' else 'stmt'
| if 'expr' then 'stmt'

on seeing the input 'if', we cannot immediately tell which production to choose to expand 'stmt'.

* In general, if $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$ are two A-productions, and the input begins with a non-empty string derived from α, we do not know whether to expand A to $\alpha\beta_1$ or $\alpha\beta_2$. However, we may defer the decision by expanding A to $\alpha A'$.
* Then, after seeing the input derived from α, we expand A' to β_1 or to β_2. That is, left-factored, the original productions become : $A \rightarrow \alpha A'$

$A' \rightarrow \beta_1 \mid \beta_2$

Algorithm : Left factoring a grammar

INPUT : Grammar G

OUTPUT : An equivalent left-factored grammar.

METHOD : For each nonterminal A, find the longest prefix α common to two or more of its alternatives. If $\alpha \neq \epsilon$, i.e., there is a nontrivial common prefix - replace all of the A-productions $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \mid \ldots \mid \alpha\beta_n \mid \gamma$, where γ represents all alternatives

that do not begin with α, by

$$A \rightarrow \alpha A' \mid \gamma$$

$$A' \rightarrow \beta_1 \mid \beta_2 \mid \ldots \mid \beta_n$$

Here, A' is a new nonterminal. Repeatedly apply this transformation until no two alternatives for a nonterminal have a common prefix.

4.3.5 Non-Context-Free Language Constructs :-

* A few syntactic constructs found in typical programming languages cannot be specified using grammars alone.
* Here, we consider two of these constructs, using simple abstract languages to illustrate the difficulties.

Example:- The language in this example abstracts the problem of checking that identifiers are declared before they are used in a program.

* The language consists of strings of the form wcw, where the first 'w' represents the declaration of an identifier w, 'c' represents an intervening program fragment, and the second 'w' represents the use of the identifier.
* The abstract language is $L_1 = \{wcw \mid w \text{ is in } (a|b)^*\}$.
* L_1 consists of all words composed of a repeated string of a's and b's separated by c, such as aabcaab.
* It is difficult to prove that the non-context-freedom of L_1 directly using non-context-freedom of programming languages like C or Java. Because C and Java does not distinguish among identifiers that are different character strings.
* Instead, all identifiers are represented by a token such as 'id' in the grammar. In a compiler for such a language, the semantic-analysis phase checks that identifiers are declared before they are used.

4.4 Top-Down Parsing:-

* The process of constructing a parse tree for the input string, starting from the root and creating the nodes of the parse tree in preorder in depth-first search manner is called top-down parsing technique.
* The top-down parsing can be viewed as an attempt to find a leftmost derivation for an input string and constructing the parse tree for that derivation.
* The parser that uses this approach is called top-down parser.
* Since the parsing starts from top (i.e. root) down to the leaves, it is called top-down parsing.

Example 1:- Show the top-down parsing process for the string id + id * id for the grammar:

$E \rightarrow E+E$

$E \rightarrow E*E$

$E \rightarrow (E)$

$E \rightarrow id$

Solution:- id + id * id is obtained by the grammar by applying leftmost derivations.

$E \underset{lm}{\Rightarrow} E+E$

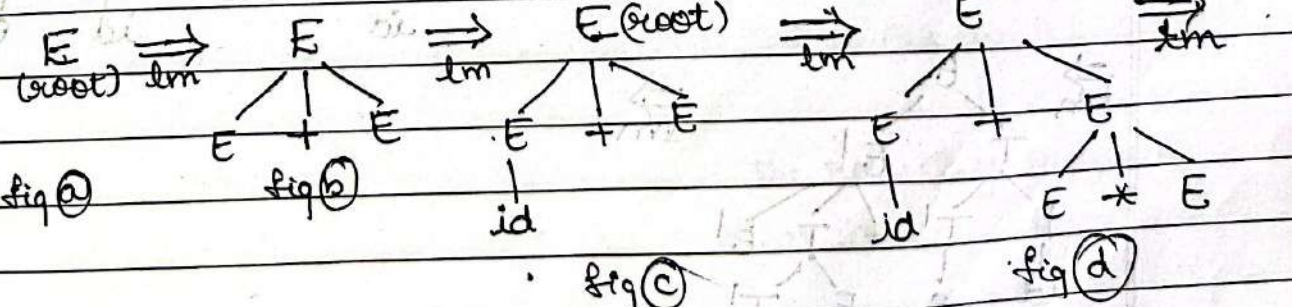
$\Rightarrow id+E$

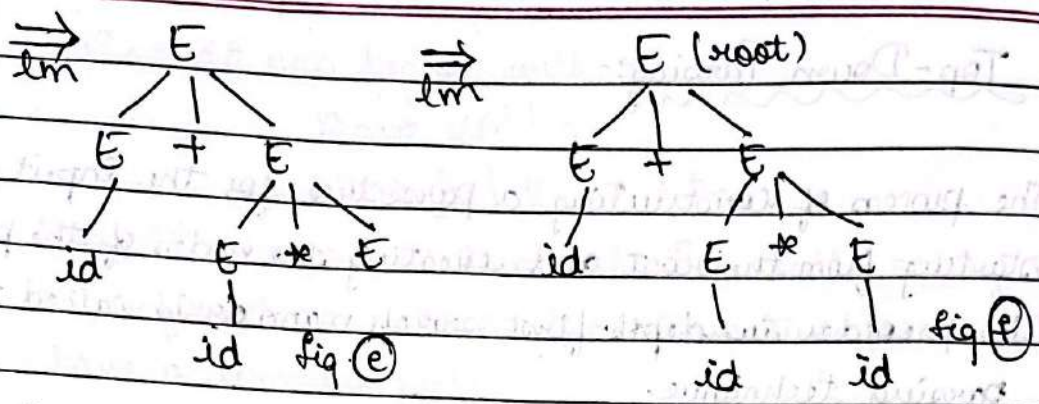
$\Rightarrow id+E*E$

$\Rightarrow id+id*E$

$\Rightarrow id+id*id$

} This derivation can be written in the form of a parse tree from the start symbol using top-down approach as shown:





Steps as follows:-

Initial:- Start from root node E. (fig (a))

Step 1: Replace E with E+E using $E \rightarrow E+E$. (fig (b))

Step 2: Replace E with id using $E \rightarrow id$. (fig (c))

Step 3:- Replace E with E*E using $E \rightarrow E*E$. (fig (d))

Step 4: Replace E with id using $E \rightarrow id$. (fig (e))

Step 5: Replace E with id using $E \rightarrow id$. (fig (f))

Example 2:- Show the top-down parsing process for the string id + id * id for the grammar:

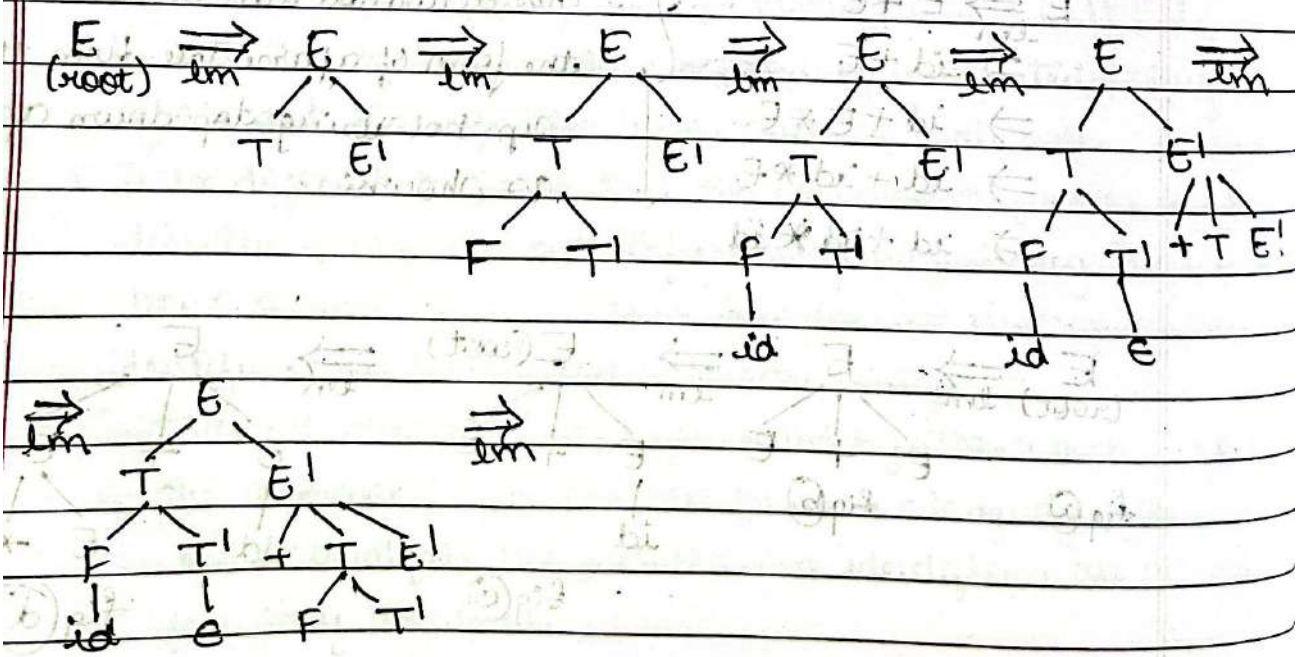
$E \rightarrow TE'$

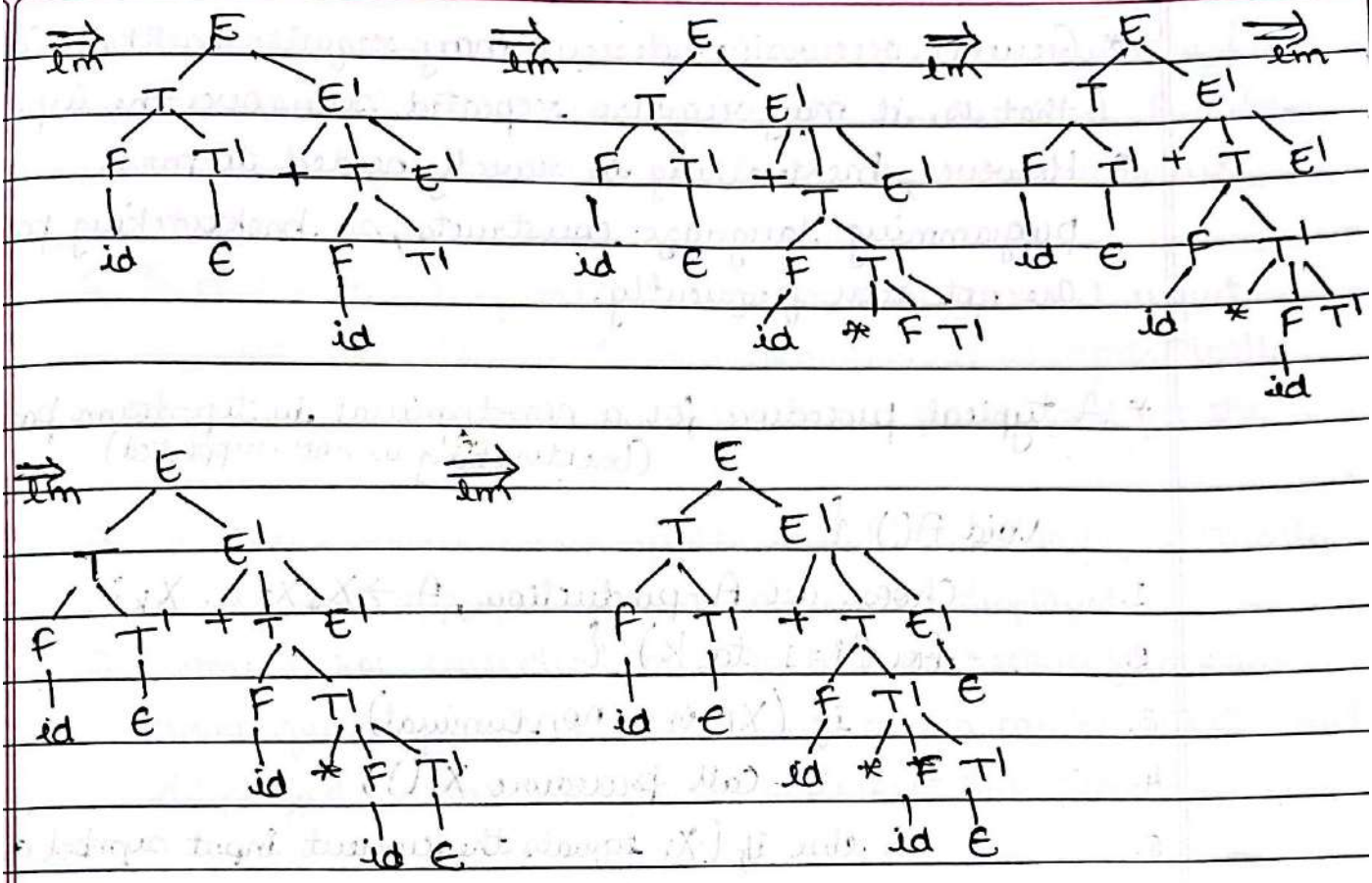
$E' \rightarrow +TE' \mid \epsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \epsilon$

$F \rightarrow (E) \mid id$





4.4.1 Recursive-Descent Parsing:-

* A recursive descent parser is a top down parser in which parse tree is constructed from the top starting from root node and selecting the productions from left to right (if two or more alternative productions exists).
* For every non-terminal there exists a recursive procedure and the right hand of the production of that non-terminal is implemented as the body of the procedure.
* The sequence of terminals and non-terminals on the right hand side of the production correspond to matching with input symbols and calls to other procedures while selecting the alternate production is implemented using switch or if-statements.
* Thus, the syntax or structure of the resulting program closely mirrors that of the grammar it recognizes.

* General recursive-descent may require backtracking; that is, it may require repeated scans over the input.
* However, backtracking is rarely needed to parse programming language constructs, so backtracking parsers are not seen frequently.

* A typical procedure for a non-terminal in top-down parser (backtracking is not supported)

```
void A() {
1.    Choose an A-production, A → X1 X2 ... Xk;
2.    for (i=1 to k) {
3.        if (Xi is a nonterminal)
4.            call procedure Xi();
5.        else if (Xi equals the current input symbol a)
6.            advance the input to the next symbol;
7.        else /* an error has occurred */;
8.    }
}
```

Working: The recursive descent parser works as shown below:

1. Execution starts from the function that corresponds to the start symbol of the grammar.
2. If the body of the function scans the entire input string, then parsing is successful. Otherwise, the input string is not proper and parsing halts.
3. Each non-terminal is associated with a parsing procedure or a function that can recognize any sequence of tokens generated by that non-terminal.
4. Within the function, both non-terminals and terminals are matched.

5. To match the non-terminal A, we call the function / procedure A which corresponds to non-terminal A. These calls may be recursive and hence the name recursive descent parser.

6. To match the terminal 'a', we compare current input symbol with 'a'. If there is a match, it is syntactically correct and we increment the input pointer and get the next token.

7. If the current input symbol is not 'a', it is syntactically wrong and appropriate error message is displayed.

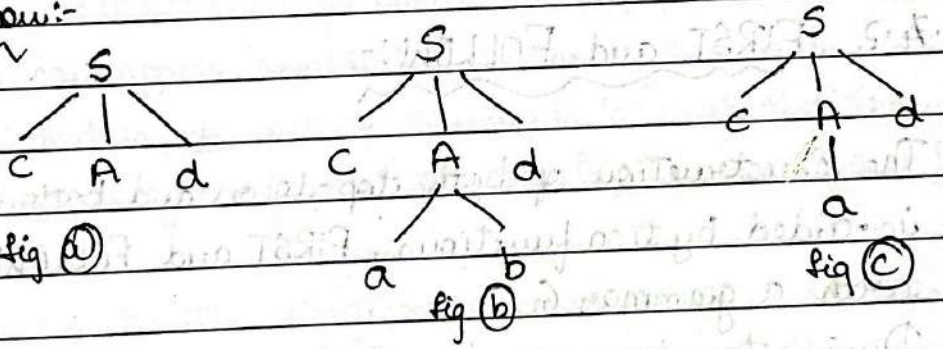
8. Some error correction may be done to recover from each error quickly so that subsequent errors can be detected and displayed so that the user can correct the programs.

Example:- Consider the grammar:

$$S \longrightarrow cAd$$
$$A \longrightarrow ab \mid a$$

Construct a parse tree using top-down for input string w=cad.

Solution:-



Step 1:- Begin with a tree consisting of a single node labeled S, and the input pointer pointing to c, the first symbol of w. S has only one production, so we use it to expand S and obtain the tree in fig (a). The leftmost leaf, labeled c, matches the first symbol of input w, so we advance the input pointer to 'a', the second symbol of 'w', and consider the next leaf, labeled A.

Step 2:- Expand A using the first alternative A → ab to obtain the tree fig (b). We have a match for the second input symbol, 'a', so we advance the input pointer to 'd', the third input symbol, and compare 'd' against the next leaf, labeled 'b'. Since 'b' does not match 'd', we report failure and go back to 'A' to see whether there is another alternative for A that has not been tried.

Step 3:- In going back to A, we must reset the input pointer to position 2, the position it had when we first came to A, which means that the procedure for A must store the input pointer in a local variable.

Step 4:- Second alternative for A produces the tree fig (c). The leaf 'a' matches the second symbol of 'w' and the leaf 'd' matches the third symbol. Since we have produced a parse tree for 'w', we halt and announce successful completion of parsing.

4.4.2 FIRST and FOLLOW:-

* The construction of both top-down and bottom-up parsers is aided by two functions, FIRST and FOLLOW, associated with a grammar G.
* During top-down parsing, FIRST and FOLLOW allow us to choose which production to apply, based on the next input symbol.
* During panic-mode error recovery, sets of tokens produced by FOLLOW can be used as synchronizing tokens (valid tokens).

* Define FIRST(α), where α is any string of grammar symbols, to be the set of terminals that begin strings derived from α. If $\alpha \overset{*}{\Rightarrow} \epsilon$, then ϵ is also in FIRST(α).

Example:- In the below figure, $A \overset{*}{\Rightarrow} c\beta$, so c is in FIRST(A).

* To use FIRST during predictive parsing, consider two A-productions $A \to \alpha \mid \beta$, where FIRST(α) and FIRST(β) are disjoint sets. Now choose between these A-productions by looking at the next input symbol 'a', since 'a' can be in at most one of FIRST(α) and FIRST(β), not both.
For instance, if 'a' is in FIRST(β) choose the production $A \to \beta$.

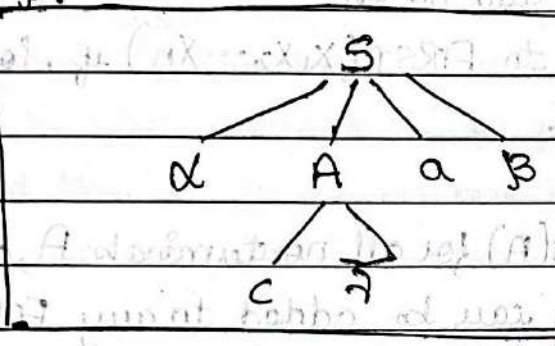


Fig 4.15: Terminal c is in FIRST(A) and α is in FOLLOW(A)

* Define FOLLOW(A), for nonterminal A, to be the set of terminals 'a' that can appear immediately to the right of A in some sentential form; that is, the set of terminals 'a' such that there exists a derivation of the form $S \overset{*}{\Rightarrow} \alpha A a \beta$, for some α and β in above fig.

* If 'A' can be the rightmost symbol in some sentential form, then \$ is in FOLLOW(A); [\$ is a special 'endmarker' symbol]

* To compute FIRST(X) for all grammar symbols X, apply the following rules until no more terminals or ϵ can be added to any FIRST set.

1. If X is a terminal, then FIRST(X) = {X}.
2. If X is a non-terminal and $X \to Y_1 Y_2 \ldots Y_k$ is a production for some $k \geq 1$, then place 'a' in FIRST(X) if for some i, a is in FIRST(Y_i), and ϵ is in all of FIRST(Y_1)... FIRST(Y_{i-1}); If ϵ is

in FIRST(Y_j) for all $j = 1, 2, \ldots, k$, then add ϵ to FIRST(X).
Eg: everything in FIRST(Y_1) is surely in FIRST(X). If Y_1 does not derive ϵ, then we add nothing more to FIRST(X), but if $Y_1 \overset{*}{\Rightarrow} \epsilon$, then we add FIRST($Y_2$), and so on.

3. If $X \to \epsilon$ is a production, then add ϵ to FIRST(X).

* To compute FIRST for any string $X_1 X_2 \cdots X_n$ as follows:-

1. Add to FIRST($X_1 X_2 \cdots X_n$) all non-ϵ symbols of FIRST(X_1).
2. Also add the non-ϵ symbols of FIRST(X_2), if ϵ is in FIRST(X_1);
3. Add the non-ϵ symbols of FIRST(X_3), if ϵ is in FIRST(X_1) and FIRST(X_2); and so on.
4. Finally, add ϵ to FIRST($X_1 X_2 \cdots X_n$) if, for all i, ϵ is in FIRST(X_i).

* To compute FOLLOW(A) for all nonterminals A, apply the following rules until nothing can be added to any FOLLOW set.

1. Place \$ in FOLLOW(S), where S is the start symbol, and \$ is the input right endmarker.
2. If there is a production $A \to \alpha B\beta$, then everything in FIRST(β) except ϵ is in FOLLOW(B).
3. If there is a production $A \to \alpha B$, or a production $A \to \alpha B\beta$, where FIRST(β) contains ϵ, then everything in FOLLOW(A) is in FOLLOW(B).

Example:- Consider non-left recursive grammar.

$E \to TE'$
$E' \to +TE' \mid \epsilon$
$T \to FT'$
$T' \to *FT' \mid \epsilon$
$F \to (E) \mid id$

Then:

1. FIRST(F) = FIRST(T) = FIRST(E) = {(, id}
2. FIRST(E') = {+, ε}
3. FIRST(T') = {*, ε}
4. FOLLOW(E) = FOLLOW(E') = {), $}
5. FOLLOW(T) = FOLLOW(T') = {+,), $}.
6. FOLLOW(F) = {+, *,), $}.

4.4.3 LL(1) Grammars:-

* Predictive parsers, i.e., recursive-descent parsers needing no backtracking can be constructed for a class of grammars called LL(1).
* The first 'L' in LL(1) stands for scanning the input from left to right, second 'L' for producing a leftmost derivation, and the '1' for using one input symbol of look ahead at each step to make parsing action decision.

* The class of LL(1) grammars is rich enough to cover most programming constructs, although care is needed in writing a suitable grammar for the source language.
Example, no left-recursive or ambiguous grammar can be LL(1).
* A grammar G is LL(1) if and only if whenever $A \to \alpha \mid \beta$ are two distinct productions of G, the following conditions hold:
1. For no terminal 'a' do both α and β derive strings beginning with 'a'.
2. At most one of α and β can derive the empty string.
3. If $\beta \overset{*}{\Rightarrow} \epsilon$, then α does not derive any string beginning with a terminal in FOLLOW(A). Likewise, if $\alpha \overset{*}{\Rightarrow} \epsilon$, then β does not derive any string beginning with a terminal in FOLLOW(A)

* The first two conditions are equivalent to the statement that FIRST(α) and FIRST(β) are disjoint sets. The third condition

is equivalent to stating that if ϵ is in FIRST(β), then FIRST(α) and FOLLOW(A) are disjoint sets, and likewise if ϵ is in FIRST(γ).

* Predictive parsers can be constructed for LL(1) grammars since the proper production to apply for a non-terminal can be selected by looking only at the current input symbol.
* Flow-of-control constructs, with their distinguishing keywords, generally satisfy the LL(1) constraints.
* For instance, if we have the productions:

```
stmt → if (expr) stmt else stmt
     | while (expr) stmt
     | {stmt_list}
```

then the keywords 'if', 'while', and the symbol '{' tell us which alternative is the only one that could possibly succeed if we are to find a statement.

* The algorithm below collects the information from FIRST and FOLLOW sets into a predictive parsing table M[A,a], a two-dimensional array, where 'A' is a nonterminal, and 'a' is a terminal or the symbol $, the input endmarker.
* The algorithm is based on the following idea: the production $A \to \alpha$ is chosen if the next input symbol 'a' is in FIRST(α). The only complication occurs when $\alpha = \epsilon$ or, more generally, $\alpha \overset{*}{\Rightarrow} \epsilon$. In this case, we should again choose $A \to \alpha$, if the current input symbol is in FOLLOW(A), or if the $ on the input has been reached and $ is in FOLLOW(A).

Algorithm: Construction of a predictive parsing table.

INPUT: Grammar G.

OUTPUT: Parsing table M.

METHOD: For each production $A \to \alpha$ of the grammar, do the following:

1. For each terminal 'a' in FIRST(α), add $A \to \alpha$ to M[A, a].
2. If ϵ is in FIRST(α), then for each terminal 'b' in FOLLOW(A), add $A \to \alpha$ to M[A, b]. If ϵ is in FIRST(α) and \$ is in FOLLOW(A), add $A \to \alpha$ to M[A, \$] as well.

* If after performing the above, there is no production at all in M[A, a], then set M[A, a] to 'error' (which is normally represented by an empty entry in the table).

Example:- Consider the grammar given and the above algorithm and produce the parse table.

$E \to T E'$

$E' \to +TE' \mid \epsilon$

$T \to FT'$

$T' \to *FT' \mid \epsilon$

$F \to (E) \mid id$

Solution:- The parsing table for the above grammar as follows:

Non-Terminal	INPUT SYMBOL					
	id	+	*	(	)	\$
E	$E \to TE'$			$E \to TE'$		
E'		$E' \to +TE'$			$E' \to \epsilon$	$E' \to \epsilon$
T	$T \to FT'$			$T \to FT'$		
T'		$T' \to \epsilon$	$T' \to *FT'$		$T' \to \epsilon$	$T' \to \epsilon$
F	$F \to id$			$F \to (E)$		

Explanation next page

* Consider the production, $E \to TE'$. Since
$FIRST(TE') = FIRST(T) = \{ (, id \}$
this production is added to $M[E, (]$ and $M[E, id]$
* Production $E' \to +TE'$ is added to $M[E', +]$ since
$FIRST(+TE') = \{+\}$. Since $FOLLOW(E') = \{), \$ \}$,
production $E' \to \epsilon$ is added to $M[E',)]$ and $M[E', \$]$.

* This algorithm can be applied to any grammar G to produce a parsing table M.
* For every LL(1) grammar, each parsing-table entry uniquely identifies a production or signals an error.

Example 2:- Consider the following grammar that has no LL(1), but it abstracts the dangling-else problem.

$S \to iEtSS' \mid a$
$S' \to eS \mid \epsilon$
$E \to b$

Solution:- The parsing table for the above grammar as follows:

Non-terminal	INPUT SYMBOL					
	a	b	e	i	t	$
S	$S \to a$			$S \to iEtSS'$		
S'			$S' \to \epsilon$ $S' \to eS$			$S' \to \epsilon$
E		$E \to b$				

* The entry for $M[S', e]$ contains both $S' \to eS$ and $S' \to \epsilon$.
* The grammar is ambiguous and the ambiguity is manifested by a choice in what production to use when an 'e' (else) is seen.
* We can resolve this ambiguity by choosing $S' \to eS$. This choice corresponds to associating an 'else' with closest previous 'then'.
* $S' \to \epsilon$ would prevent 'e' from ever being put on stack or removed from input, and is surely wrong.

4.4.4 Non-recursive Predictive Parsing :-

* A nonrecursive predictive parser can be built by maintaining a stack explicitly, rather than implicitly via recursive calls.
* The parser mimics a leftmost derivation. If 'w' is the input that has been matched so far, then the stack holds a sequence of grammar symbols α such that,

$$S \xRightarrow{*}_{lm} w\alpha$$

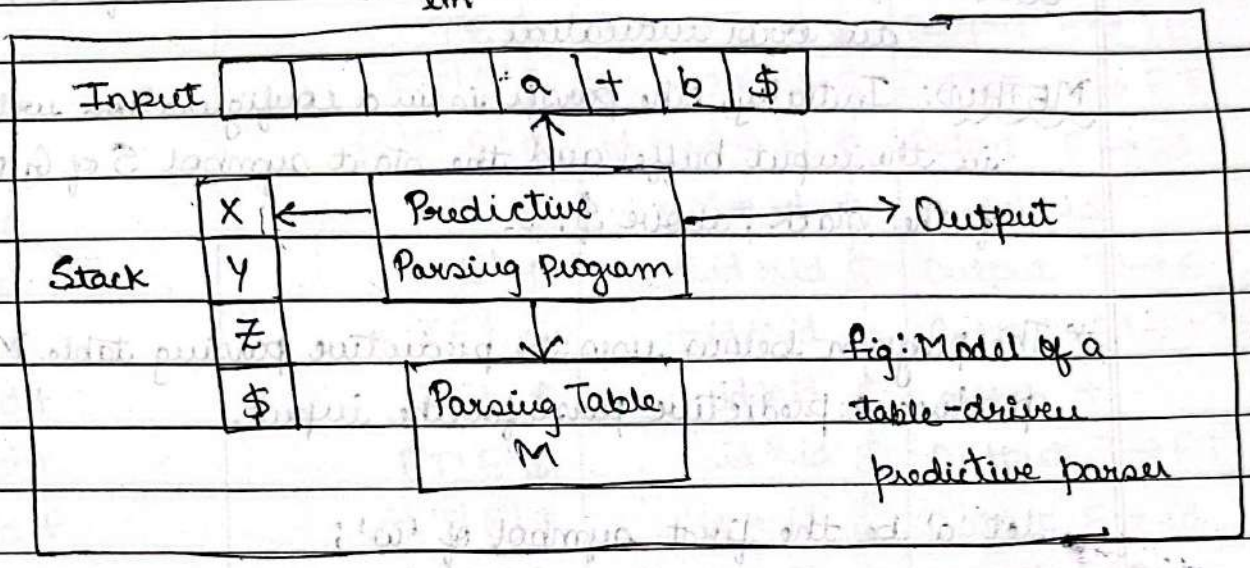


fig: Model of a table-driven predictive parser

* The table-driven parser has an input buffer, a stack containing a sequence of grammar symbols, a parsing table constructed by the algorithm (construction of predictive parsing table) and the output stream.
* The input buffer contains the string to be parsed, followed by the endmarker $. We reuse the symbol $ to mark the bottom of the stack, which initially contains the start symbol of the grammar on top of $.
* The parser is controlled by a program that considers X, the symbol on top of the stack, and 'a', the current input symbol.
* If X is a nonterminal, the parser chooses an X-production by consulting entry M[X, a] of the parsing table M. Otherwise, it checks for a match between the terminal X and current input symbol 'a'.

* The behavior of the parser can be described in terms of its 'configurations', which gives the stack contents and the remaining input.

Algorithm: Table-driven predictive parsing.

INPUT: A string 'w' and a parsing table M for grammar G.
OUTPUT: If 'w' is in L(G), a leftmost derivation of 'w'; otherwise an error indication.
METHOD: Initially, the parser is in a configuration with w$ in the input buffer and the start symbol S of G on top of the stack, above $.

* The program below uses the predictive parsing table M to produce a predictive parse for the input.

```
let 'a' be the first symbol of 'w';
let X be the top stack symbol;
while (X ≠ $) { /* stack is not empty */
    if (X = a) pop the stack and let 'a' be the next symbol of 'w';
    else if (X is a terminal) error();
    else if (M[X, a] is an error entry) error();
    else if (M[X, a] = X → Y1 Y2 ... Yk) {
        output the production X → Y1 Y2 ... Yk;
        pop the stack;
        push Yk, Yk-1, ..., Y1 onto the stack, with Y1 on top;
    }
    let X be the top stack symbol;
}
```

fig: Predictive parsing Algorithm.

Example:- Consider the given grammar, for the input id+id*id, the apply the nonrecursive predictive parsing algorithm

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \epsilon$ $\quad\quad$ $T' \rightarrow *FT' \mid \epsilon$

$T \rightarrow FT'$ $\quad\quad$ $F \rightarrow (E) \mid id$

Solution:- Moves made by predictive parser on input id+id*id

Matched	Stack	INPUT	ACTION
	$E\$$	id+id*id$	
	$TE'\$$	id+id*id$	Output $E \rightarrow TE'$
	$FT'E'\$$	id+id*id$	Output $T \rightarrow FT'$
	$id\,T'E'\$$	id+id*id$	Output $F \rightarrow id$
id	$T'E'\$$	+id*id$	match id
id	$E'\$$	+id*id$	Output $T' \rightarrow \epsilon$
id	$+TE'\$$	+id*id$	Output $E' \rightarrow +TE'$
id+	$TE'\$$	id*id$	match +
id+	$FT'E'\$$	id*id$	Output $T \rightarrow FT'$
id+	$id\,T'E'\$$	id*id$	Output $F \rightarrow id$
id+id	$T'E'\$$	*id$	match id
id+id	$*FT'E'\$$	*id$	Output $T' \rightarrow *FT'$
id+id*	$FT'E'\$$	id$	match *
id+id*	$id\,T'E'\$$	id$	Output $F \rightarrow id$
id+id*id	$T'E'\$$	$	match id
id+id*id	$E'\$$	$	Output $T' \rightarrow \epsilon$
id+id*id	$\$$	$	Output $E' \rightarrow \epsilon$

* Note that the sentential forms in this derivation correspond to the input that has already been matched (in column MATCHED) followed by the stack contents.
* The top of the stack is to the left; when we consider bottom-up parsing it will be more natural to show the top of the stack to right.
* The input pointer points to the leftmost symbol of the string in the INPUT column.

4.4.5 Error Recovery in Predictive Parsing :-

* An error is detected during predictive parsing when
  (a) the terminal on top of the stack does not match the next input symbol, or
  (b) the non-terminal A is on the top of the stack, 'a' is the next input symbol, and M[A,a] is 'error' (i.e., the parsing table entry is empty).

* The error recovery is done using panic mode and phrase-level recovery.

1. Panic mode :- In this approach, error recovery is done by skipping symbols from the input until a token matches with synchronizing tokens.
   * The synchronizing tokens are selected such that the parser should quickly recover from the errors that are likely to occur in practice. Some of the recovery techniques are as follows:

a. As a starting point, place all symbols in FOLLOW(A) into the synchronizing set for nonterminal A. If we skip tokens until an element of FOLLOW(A) is seen and pop A from the stack, it is likely that parsing can continue.

b. It is not enough to use FOLLOW(A) as the synchronizing set for A. For example, if semicolons terminate statements, as in C, then keywords that begin statements may not appear in the FOLLOW set of the nonterminal representing expressions. A missing semicolon after an assignment may therefore result in the keyword beginning the next statement being skipped.

c. If we add symbols in FIRST(A) to the synchronizing set for nonterminal A, then it may be possible to resume parsing according to A if a symbol in FIRST(A) appears in the input.

d. If a nonterminal can generate the empty string, then the production deriving ε can be used as a default. Doing so may postpone some error detection, but cannot cause an error to be missed. This approach reduces the number of nonterminals that have to be considered during error recovery.

e. If a terminal on top of the stack cannot be matched, a simple idea is to pop the terminal, issue a message saying that the terminal was inserted, and continue parsing. This approach takes the synchronizing set of token to consist of all other tokens.

2. Phrase-level Recovery:- This approach is implemented by filling in the blank entries in the predictive parsing table with pointers to error routines.

* These routines may change, insert or delete symbols on the input and issue appropriate error messages. They may also pop from the stack.
* Alteration of stack symbols or the pushing of new symbols onto the stack is questionable for several reasons:
  a. The steps carried out by the parser might then not correspond to the derivation of any word in the language at all.
  b. We must ensure that there is no possibility of an infinite loop. Checking that any recovery action eventually results in an input symbol being consumed is good way to protect against such loops.

4.5 Bottom-Up Parsing:-

* A bottom-up parse corresponds to the construction of a parse tree for an input string beginning at the leaves (bottom) and working up towards the root (top).
* It is convenient to describe parsing as the process of building parse trees, although a front end may in fact carry out a translation directly without building an explicit tree.
* The sequence of tree snapshots below illustrates a bottom-up parse of the token stream id * id, wrt to expression grammar (4.1).

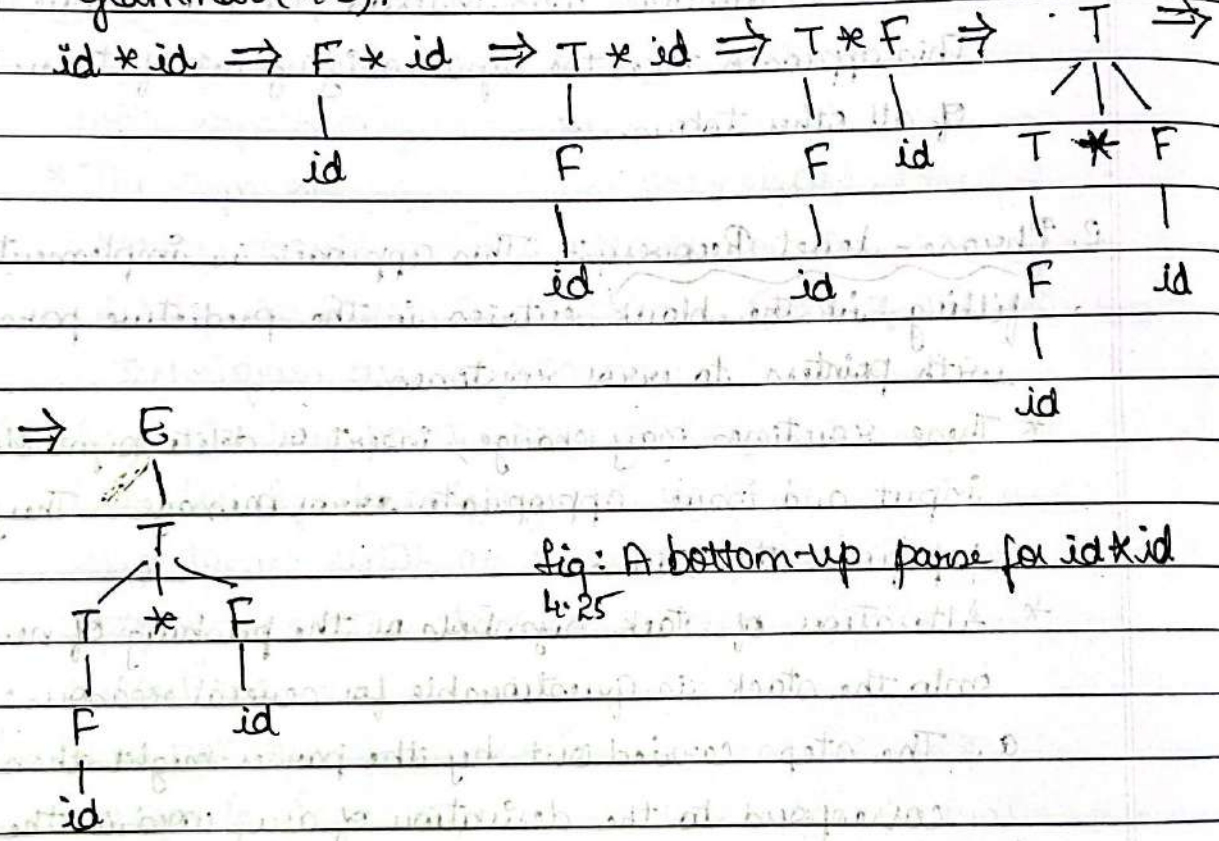


fig: A bottom-up parse for id * id

4.25

4.5.1 Reductions:-

* During bottom-up parsing, the given string of terminals are reduced to the start symbol. The process of searching for right-hand side of the production in a string consisting of terminals and/or non-terminals and replacing it with corresponding

left hand side of the production is called reduction.

Example :- Consider the following grammar:

$E \rightarrow E+T \mid T$

$T \rightarrow T*F \mid F$

$F \rightarrow (E) \mid id$

* The right most derivation to get the string id * id is as shown:

$E \Rightarrow T$	Using $E \rightarrow T$
$\Rightarrow T*F$	Using $T \rightarrow T*F$
$\Rightarrow T*id$	Using $F \rightarrow id$
$\Rightarrow F*id$	Using $T \rightarrow F$
$\Rightarrow id*id$	Using $F \rightarrow id$

Explanation:- If the sequence starts with input string id * id, and moves backwards by applying rightmost derivations in reverse we get start symbols E and the various sentential forms that we get while moving backwards are:

id * id , F * id , T * id , T * F , T , E

- The strings in this sequence are formed from the roots of all the subtrees in the snapshots.

* The sequence starts with the input string id * id. The first reduction produces F * id by reducing the leftmost id to F, using the production $F \rightarrow id$. The second reduction produces T * id by reducing F to T. Now, we have a choice between reducing the string T, which is the body of $E \rightarrow T$, and the string consisting of the second id, which is the body of $F \rightarrow id$.

* Rather than reduce T to E, the second id is reduced to F, resulting in the string T * F. This string then reduces to T. The parse completes with the reduction to T to the start symbol E. (Fig 4.25)

4.5.2 Handle Pruning:-

* Informal definition of handle:- A substring that matches the right hand side of a production (i.e., that matches the body of a production) and replacing it with equivalent nonterminal on LHS of the production to get one step in right most derivation in reverse is called the handle.

Right Sentential Form	Handle	Reducing Production
$id_1 * id_2$	id_1	$F \to id$
$F * id_2$	F	$T \to F$
$T * id_2$	id_2	$F \to id$
$T * F$	$T * F$	$T \to T * F$
T	T	$E \to T$

fig: Handles during a parse of $id_1 * id_2$

* Formal definition of handle:- If there is a right most derivation of the form $S \overset{*}{\Rightarrow} \alpha Aw \Rightarrow \alpha\beta w$ then the string β in i^{th} right sentential form $\alpha\beta w$ is the handle if there is a production of the form $A \to \beta$ such that replacing β by A in $\alpha\beta w$ results in previous $(i-1)^{th}$ right sentential form αAw. Note that the string w to the right of the handle must contain only string of terminals.

* Handle pruning:- The procen of discovering a handle in a right sentential form γ_n and reducing it to the appropriate left hand side to get the previous right sentential form γ_{n-1} is called handle pruning.

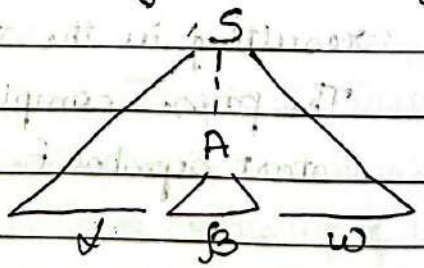


fig: A handle $A \to \beta$ in the parse tree for $\alpha\beta w$.

Example:- Consider the following right most derivation:

$$S \Rightarrow \gamma_1 \Rightarrow \gamma_2 \Rightarrow \gamma_3 \Rightarrow \cdots \Rightarrow \gamma_{n-1} \Rightarrow \gamma_n$$

Observe the following points from above right most derivation:

1. In n^{th} right sentential form γ_n, search for the handle β_n and replace it with corresponding left hand side of the production $A_n \rightarrow \beta_n$ to get $(n-1)^{th}$ right sentential form γ_{n-1}.
2. Similarly, we locate the handle β_{n-1} in γ_{n-1} and replace β_{n-1} with corresponding left hand side of the production $A_{n-1} \rightarrow \beta_{n-1}$ to get $(n-2)^{nd}$ right sentential form γ_{n-2}.
3. Repeating this process i.e., using repeated handle pruning we reduce the given string 'w' to the start symbol S and we say parsing is successful.

Example 2:- For the following grammar $S \rightarrow 0S1 | 01$, indicate the handles in the following right sentential form: 00001111.

Solution:-

Right Sentential form	Handle	Reduction Production
0000111	01	$S \rightarrow 01$
000S11	0S1	$S \rightarrow 0S1$
00S11	0S1	$S \rightarrow 0S1$
0S1	0S1	$S \rightarrow 0S1$
S		

1.5.3 Shift Reduce Parser:-

* Shift-reduce parser is an efficient way of implementing bottom up parser using explicit stack. The stack contains grammar symbols and input buffer holds the string to be parsed.
* In this parser, we start from string of terminals and get the start symbol. During parsing, always the handle will appear on top of the stack just before it is identified as the handle.

* We use $ to mark the bottom of the stack and also the right end of the input.
* We show top of the stack on the right. Initially, the stack is empty, and the string 'w' is on the input, as follows:

STACK	INPUT
$	w$

* During a left-to-right scan of the input string, the parser shifts zero or more input symbols onto the stack, until it is ready to reduce a string β of grammar symbols on top of the stack.
* It then reduces β to the head of the appropriate production.
* The parser repeats this cycle until it has detected an error or until the stack contains the start symbol and the input is empty:

STACK	INPUT
$S	$

* Upon entering this configuration, the parser halts and announces successful completion of parsing.

* The four possible actions of the shift reduce parser are:
1. Shift: Shift the next input symbol onto the top of stack.
2. Reduce: The right end of the string to be reduced must be at the top of the stack. Locate the left end of the string within the stack and decide with what nonterminal to replace the string.
3. Accept: Announce successful completion of parsing.
4. Error: Discover a syntax error and call an error recovery routine.

Example:- Show the sequence of moves made by the shift-reduce parser for the string id * id using the following grammar:

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid id$

Solution:-

STACK	INPUT	ACTION
$	$id_1 * id_2 \$$	Shift
$id	$* id_2 \$$	Reduce $F \rightarrow id$
$F	$* id_2 \$$	Reduce $T \rightarrow F$
$T	$* id_2 \$$	Shift *
$T*	$id_2 \$$	Shift id
$T* id	$\$$	Reduce $F \rightarrow id$
$T*F	$\$$	Reduce $T \rightarrow T * F$
$T	$\$$	Reduce $E \rightarrow T$
$E	$\$$	ACCEPT

*Since stack contains the start symbol and input is empty, we say parsing is successful.

* Observe the following points:

1. The parser has started from the string id * id $ in the input and it has reduced the input string to start symbol E on the stack.
2. Thus, starting from string of terminals i.e., bottom (from the leaves) it has obtained the start symbol S which appears at the top of the parse tree (root).
3. So, the shift reduce parser is called bottom-up-parser.

4.5.4 Conflicts During Shift-Reduce Parsing:-

* There are context-free grammars for which shift-reduce parsing cannot be used.
* Every shift-reduce parser for such a grammar can reach a configuration in which the parser, knowing the entire stack and also the next 'k' input symbols, cannot decide whether to shift or to reduce (shift/reduce conflict), or cannot decide which of several reductions to make (a reduce/reduce conflict).

1. Shift-reduce conflict:- During parsing, by knowing the contents of the stack and the next input symbol, if the parser cannot decide whether to shift the input the symbol on to the stack or to reduce the handle on top of the stack, it results in shift-reduce conflict.

Example:- Consider the grammar:

$$E \rightarrow E+T \mid T$$
$$T \rightarrow T*F \mid F$$
$$F \rightarrow (E) \mid id$$

- The sequence of moves made by the parser for the string id * id is:

STACK	INPUT	ACTION
$	id * id	Shift id
$id	* id	Reduce F → id
$F	* id	

* Observe that the parser has reached a state where it cannot decide whether to shift input symbol * on to the stack or to reduce using production T → F. This conflict is called shift-reduce conflict.

2. Reduce-reduce Conflicts:- During parsing, if shift-reduce parser finds a handle on top of the stack, the parser has to reduce the handle to the left hand side of the production. But, if two or more productions have same handle on right hand side of the production, the parser cannot decide which production has to be selected for reducing. This conflict is called reduce-reduce conflict.

Example: Consider the following grammar:

$E \rightarrow E+T \mid T$

$T \rightarrow T*F \mid F$

$F \rightarrow (E) \mid id$

The sequence of moves made by the parser for string id * id is:

STACK	INPUT	ACTION
$	id * id $	Shift id
$id	* id $	Reduce F → id
$F	* id $	Reduce T → F
$T	* id $	Shift *
$T*	id $	Shift id
$T* id	$	Reduce F → id
$T*F	$	

* Observe that the parser has reached a state where it cannot decide whether to reduce the handle F to T using the production $T \rightarrow F$ or to reduce the handle T*F to T using the production $T \rightarrow T*F$. This conflicts is called reduce-reduce conflict.

Question Bank (Module 3)

1. What is the role of parser? Explain the different error recovery strategies. (8M)

2. Construct the LL(1) parsing table for the following productions:
   $E \rightarrow E+T \mid T$
   $T \rightarrow T*F \mid F$
   $F \rightarrow (E) \mid id$
   (8M)

3. Write a short note on shift-reduce parsing with an example. (6M)

4. What is top down parser? What are key problems in top down parsing? (8M)

5. What is meant by handle pruning? How it helps on shift reduce parsing? List the actions of a shift reduce parser. (8M)

6. Explain the ambiguity in arithmetic expression. What is the ambiguity in parsing 2+3X4? Explain the solution for it. (8M)

7. What is shift reduce parser? Explain the conflicts that may occur during shift reduce parsing. (4M)

8. How left recursion can be eliminated from grammars? Write down the simple arithmetic expression grammar and rewrite the grammar after removing left recursion. (4M)

9. What is left factoring? Rewrite the following grammar after 'left factored' $S \rightarrow iEtS \mid iEtSeS \mid a$
   $E \rightarrow b$
   (4M)

10. Define Left-recursion grammar, eliminate left recursion from the following grammar:

$S \rightarrow aB \mid ac \mid Sd \mid Se$

$B \rightarrow bBc \mid f$

$C \rightarrow g$

(4M)

11. Consider the following context free grammar $S \rightarrow SS+ \mid SS* \mid a$ and the input string $aa+a*$

(i) Give LMD and RMD (ii) Parse tree

(iii) Is the grammar ambiguous? Why (iv) Left factor the grammar

(v) Describe the language generated by a grammar. (5M)

12. Consider the following grammar with terminals (, [,),]

$S \rightarrow TS \mid [S]S \mid)S \mid \epsilon$

$T \rightarrow (x)$

$X \rightarrow TX \mid [X]X \mid \epsilon$

(i) Construct first and follow sets (8M)

(ii) Construct its LL(1) parsing table.

(iii) Is this grammar LL(1)?

13. What are the actions of a shift-reduce parser. Design shift-reduce parser for the following grammar on the input 10201.

$S \rightarrow 0S0 \mid 1S1 \mid 2$ (6M)

14. Consider the CFG with the production set,

$E \rightarrow E+T \mid T$

$T \rightarrow TF \mid F$

$F \rightarrow F* \mid a \mid b$ (5M)

Compute : FIRST() and FOLLOW()

15. Compute: (i) FIRST() and FOLLOW()
(ii) Predictive parsing table for the given grammar. (6M)

$D \rightarrow L ; T$
$L \rightarrow L ; id \mid id$
$T \rightarrow int \mid real$

16. For the following grammar 'S → SS+ | SS* | a', indicate the handles in the following right sentential form 'SS+a*a+'.

17. What is bottom-up parser? Show the tree snapshots that are constructed for the ~~now~~ string id * id using bottom up parser.

$E \rightarrow E+T \mid T$
$T \rightarrow T*F \mid F$
$F \rightarrow (E) \mid id$

18. Show the sequence of moves made by the shift-reduce parser for the string 'id * id' using the following grammar:

$E \rightarrow E+T \mid T$
$T \rightarrow T*F \mid F$
$F \rightarrow (E) \mid id$